

WEB/APPLICATION TESTING (SIMULATED)

1. Introduction

The purpose of this Web/Application Testing exercise is to evaluate the security of web-based components within the BSF-Nutrifeed simulated ecosystem. As direct access to the organization's production web applications is not available, this assessment is conducted in a controlled lab environment designed to replicate real-world web application scenarios.

The simulated environment includes intentionally vulnerable web applications hosted on **Metasploitable 2**, representing potential web services that could exist within the BSF-Nutrifeed infrastructure. These applications are used to demonstrate common security weaknesses such as improper input validation, insecure session handling, and exposure of sensitive endpoints.

During this assessment, HTTP requests are intercepted and analyzed using **Burp Suite** to understand how data is transmitted between the client and server. The applications are further tested for common vulnerabilities including SQL injection, cross-site scripting (XSS), and file upload flaws.

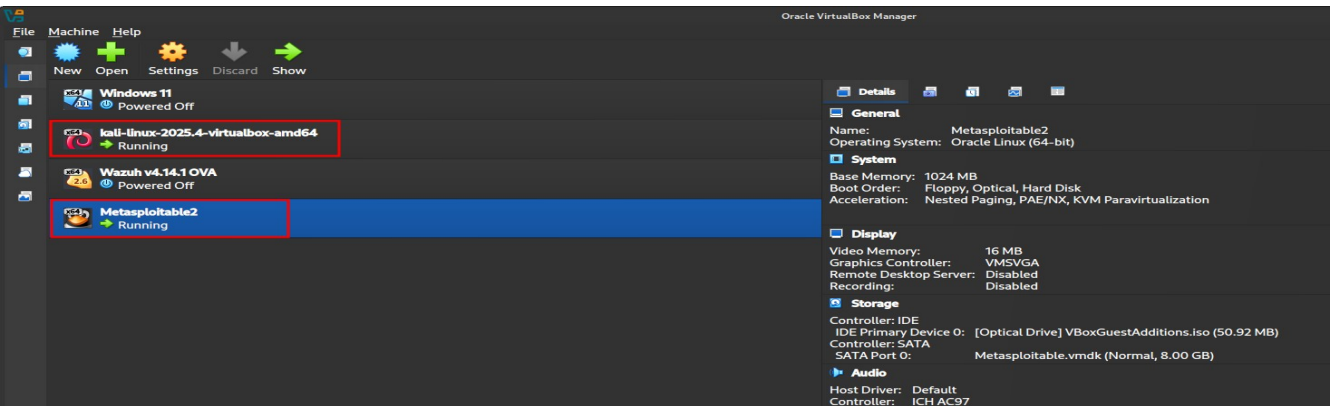
The objective of this testing is to identify weaknesses in web application security, assess their potential impact on the overall system, and provide actionable recommendations to enhance the resilience of the BSF-Nutrifeed ecosystem against web-based attacks.

2. Setting up BSF Nutri-feed lab environment

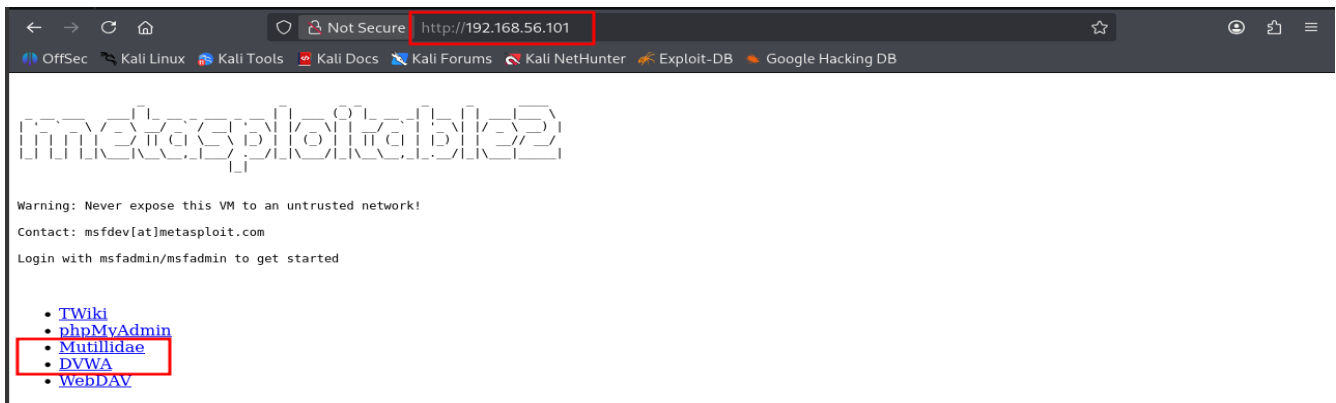
i) Lab Architecture

S/n	Lab VMs/Tools
1	Metasploitable2
2	Kali Linux VM

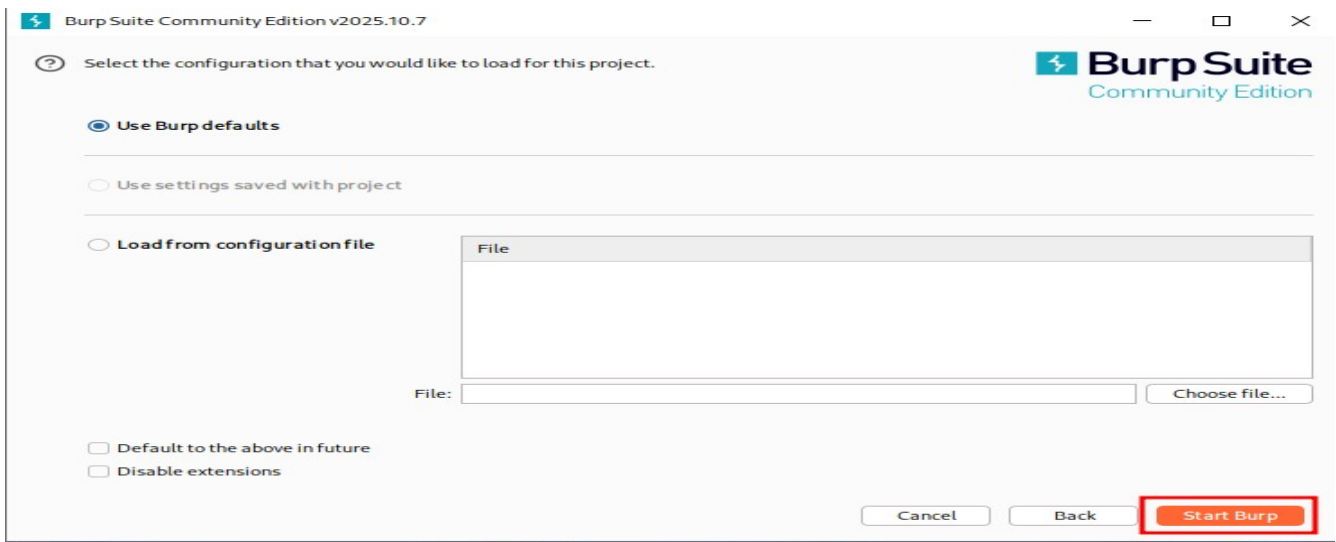
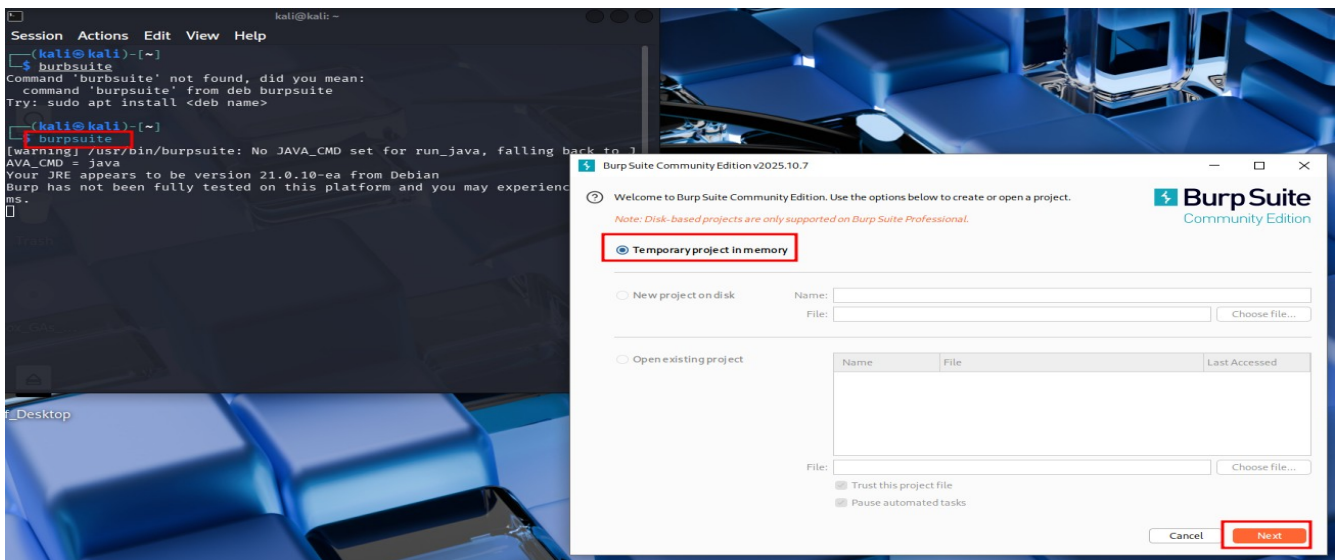
ii) Ensuring Targets are running



iii) Finding Web Application “<http://192.168.56.101>” in Kali

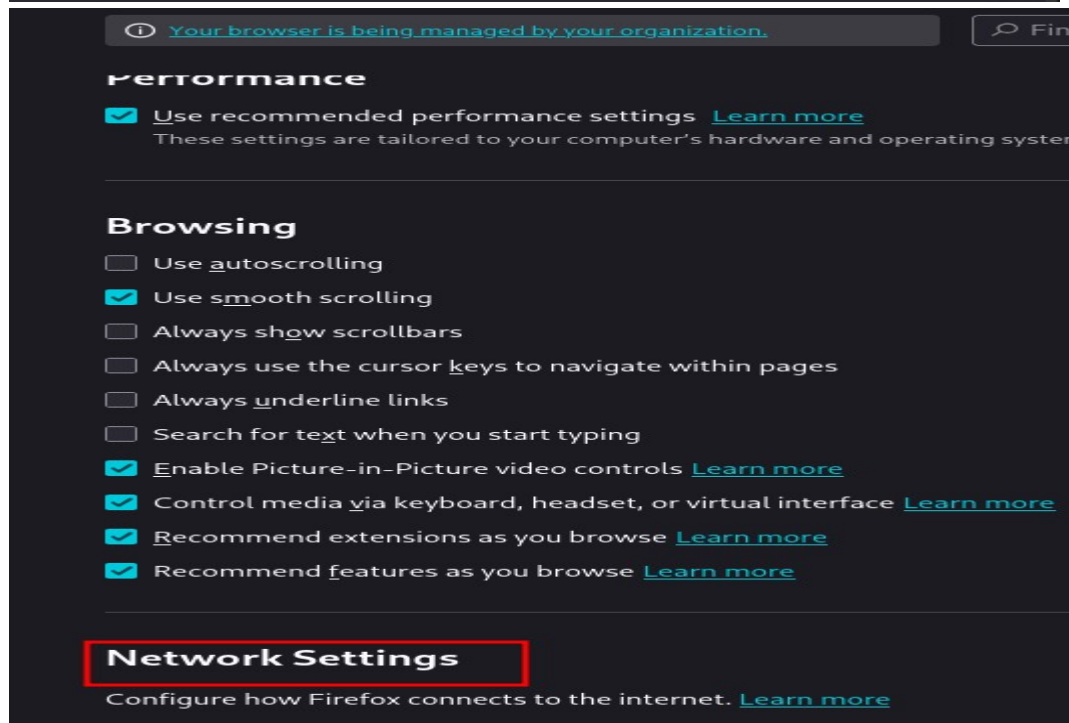
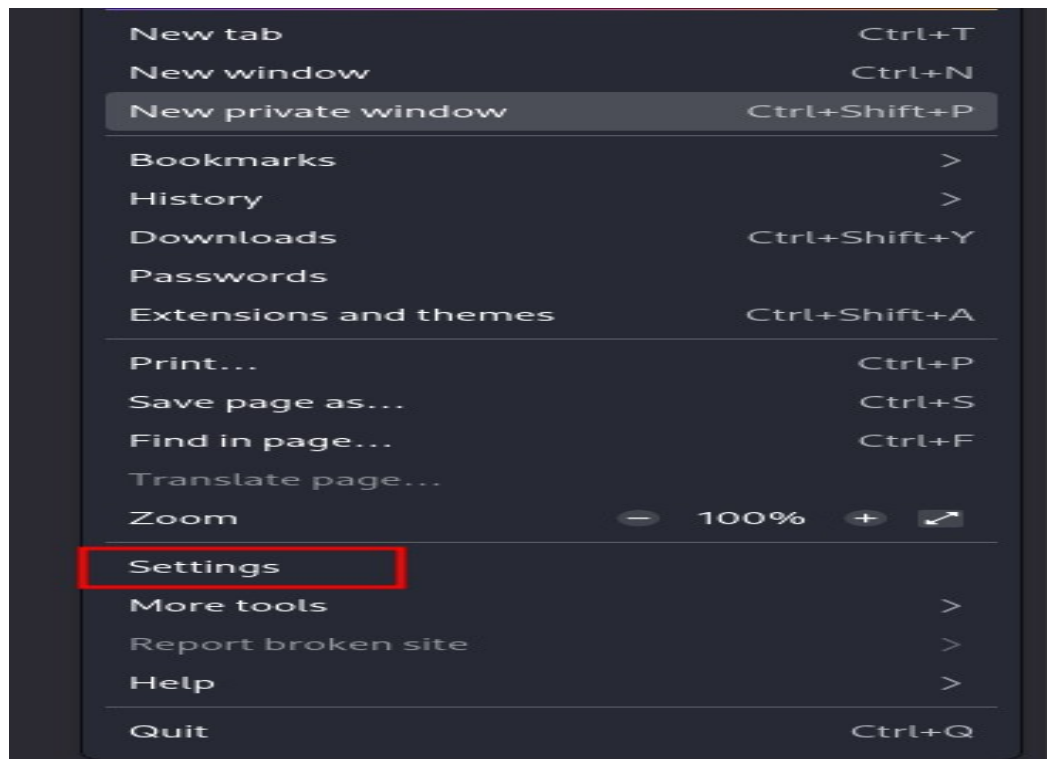


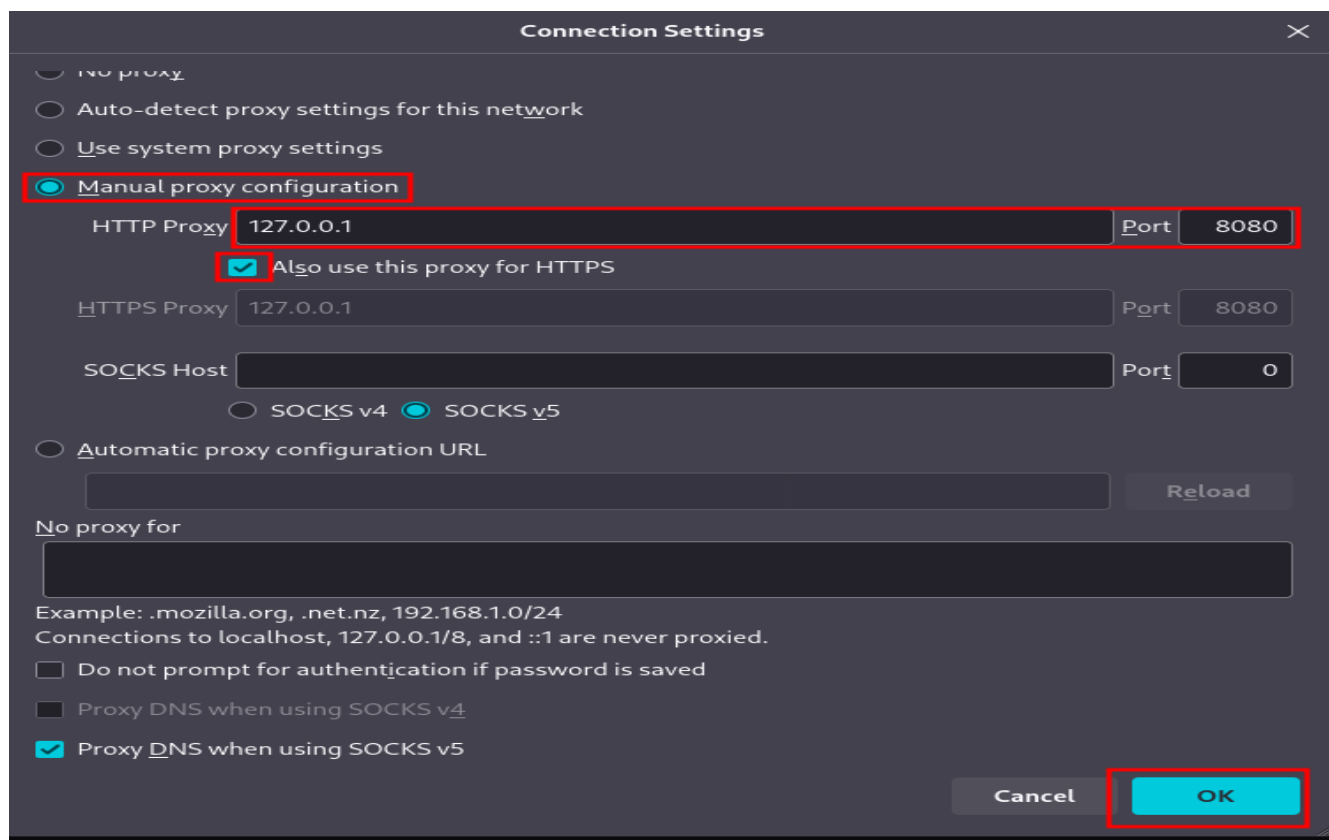
iv) Burp Suite setup



v) Configuring Browser Proxy In Firefox

Settings → Network settings → Manual Proxy →
HTTP Proxy → 127.0.0.1 : Port → 8080





* This routes traffic through Burp

Testing for Vulnerabilities

3. Capturing Requests

In Firefox Browser: (<http://192.168.56.101/dvwa/login.php>)

i) Login to DVWA page

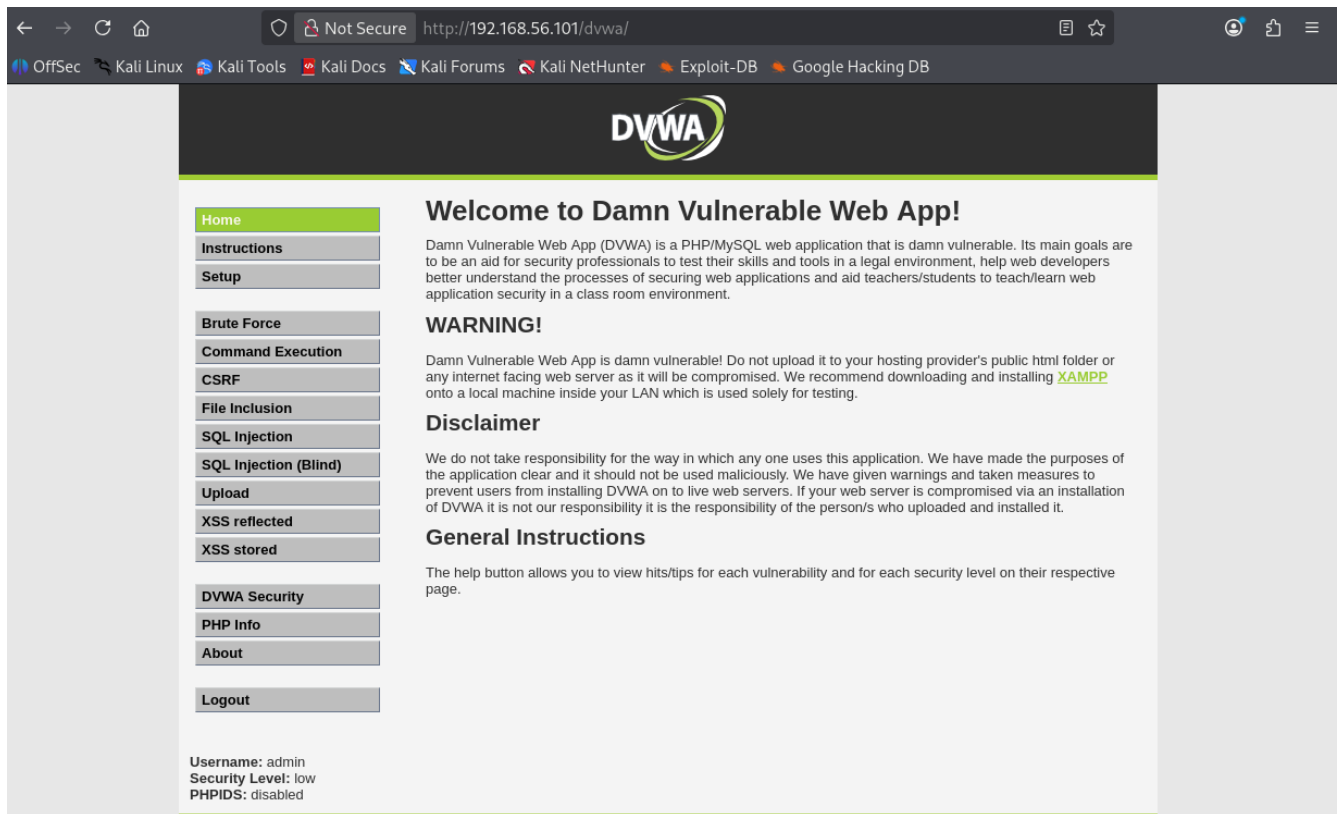


Username
admin

Password
••••••••

Login

ii) DVWA Dashboard



The screenshot shows the DVWA (Damn Vulnerable Web App) dashboard in a web browser. The browser's address bar displays `http://192.168.56.101/dvwa/`. The page features a dark header with the DVWA logo. On the left, a sidebar contains a list of navigation links: Home, Instructions, Setup, Brute Force, Command Execution, CSRF, File Inclusion, SQL Injection, SQL Injection (Blind), Upload, XSS reflected, XSS stored, DVWA Security, PHP Info, About, and Logout. The main content area is titled "Welcome to Damn Vulnerable Web App!" and includes a warning about the application's vulnerability and a disclaimer. At the bottom left, the user's login details are shown: Username: admin, Security Level: low, and PHPIDS: disabled.

Home

Instructions

Setup

Brute Force

Command Execution

CSRF

File Inclusion

SQL Injection

SQL Injection (Blind)

Upload

XSS reflected

XSS stored

DVWA Security

PHP Info

About

Logout

Username: admin
Security Level: low
PHPIDS: disabled

Welcome to Damn Vulnerable Web App!

Damn Vulnerable Web App (DVWA) is a PHP/MySQL web application that is damn vulnerable. Its main goals are to be an aid for security professionals to test their skills and tools in a legal environment, help web developers better understand the processes of securing web applications and aid teachers/students to teach/learn web application security in a class room environment.

WARNING!

Damn Vulnerable Web App is damn vulnerable! Do not upload it to your hosting provider's public html folder or any internet facing web server as it will be compromised. We recommend downloading and installing [XAMPP](#) onto a local machine inside your LAN which is used solely for testing.

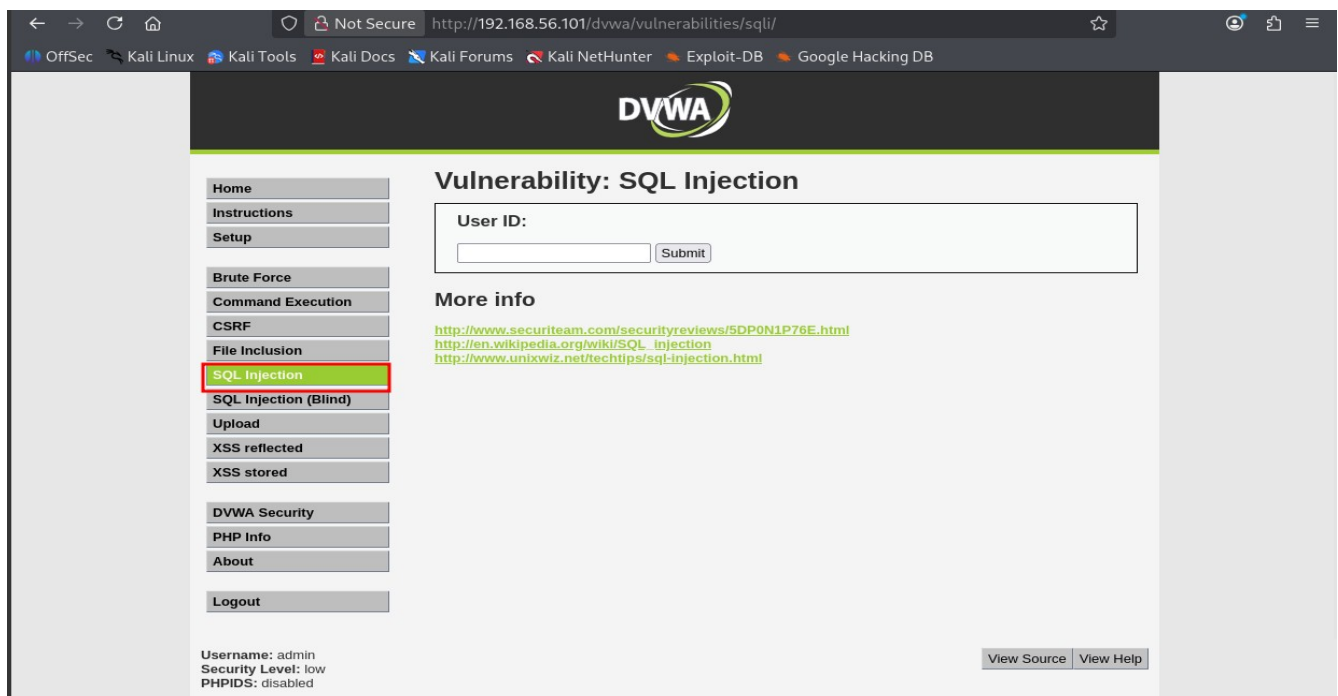
Disclaimer

We do not take responsibility for the way in which any one uses this application. We have made the purposes of the application clear and it should not be used maliciously. We have given warnings and taken measures to prevent users from installing DVWA on to live web servers. If your web server is compromised via an installation of DVWA it is not our responsibility it is the responsibility of the person/s who uploaded and installed it.

General Instructions

The help button allows you to view hits/tips for each vulnerability and for each security level on their respective page.

iii) From DVWA dashboard, go to SQL injection section



The screenshot shows the DVWA "Vulnerability: SQL Injection" page. The browser's address bar displays `http://192.168.56.101/dvwa/vulnerabilities/sqli/`. The page features a dark header with the DVWA logo. On the left, a sidebar contains a list of navigation links: Home, Instructions, Setup, Brute Force, Command Execution, CSRF, File Inclusion, SQL Injection, SQL Injection (Blind), Upload, XSS reflected, XSS stored, DVWA Security, PHP Info, About, and Logout. The "SQL Injection" link is highlighted with a red box. The main content area is titled "Vulnerability: SQL Injection" and includes a form for "User ID:" with a "Submit" button. Below the form, there is a "More info" section with three links: <http://www.securiteam.com/securityreviews/5DP0N1P76E.html>, http://en.wikipedia.org/wiki/SQL_injection, and <http://www.unixwiz.net/techtips/sql-injection.html>. At the bottom left, the user's login details are shown: Username: admin, Security Level: low, and PHPIDS: disabled. At the bottom right, there are links for "View Source" and "View Help".

Home

Instructions

Setup

Brute Force

Command Execution

CSRF

File Inclusion

SQL Injection

SQL Injection (Blind)

Upload

XSS reflected

XSS stored

DVWA Security

PHP Info

About

Logout

Username: admin
Security Level: low
PHPIDS: disabled

Vulnerability: SQL Injection

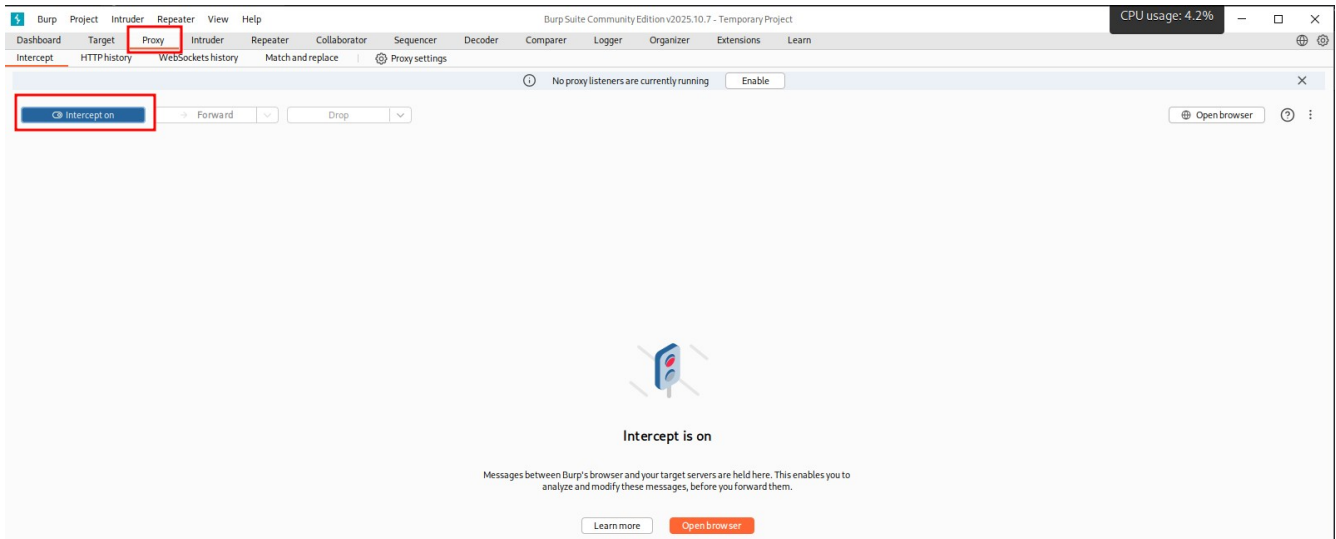
User ID:

More info

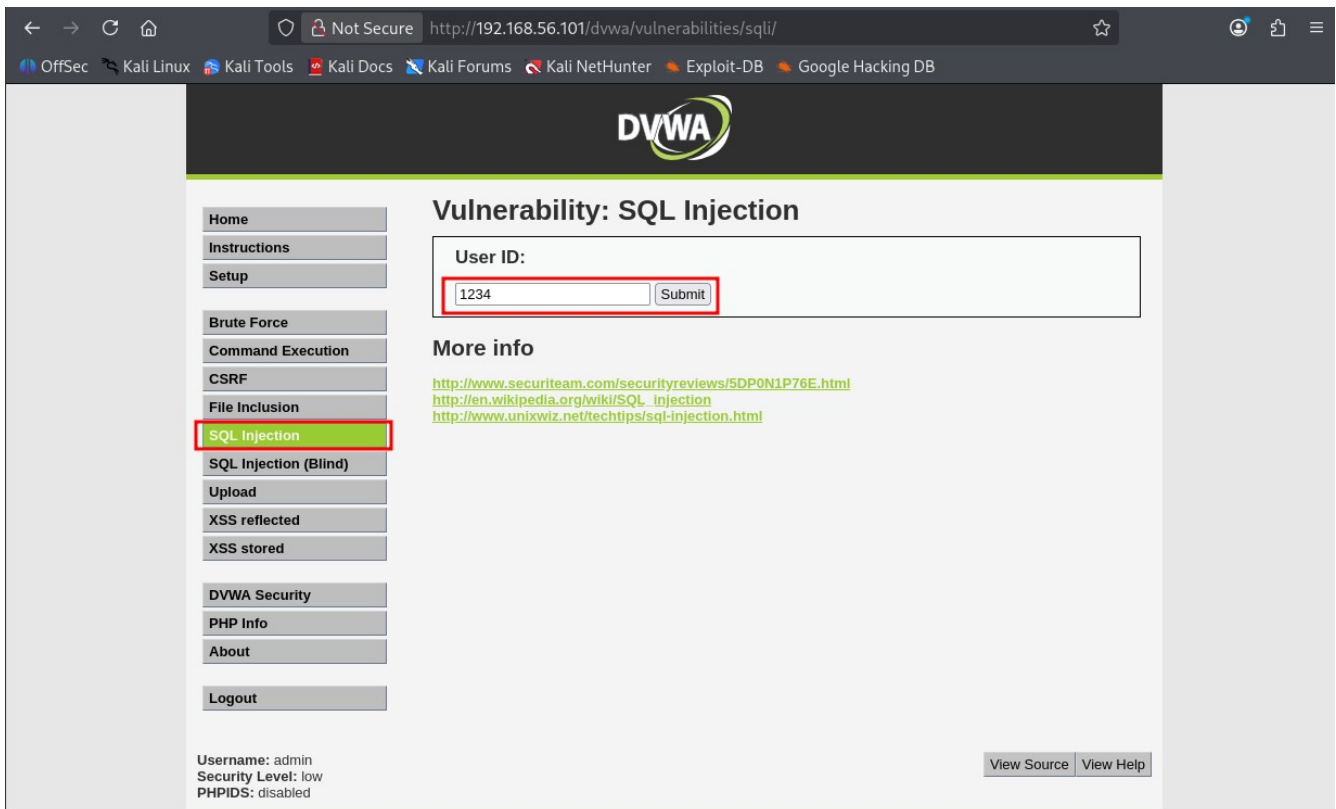
<http://www.securiteam.com/securityreviews/5DP0N1P76E.html>
http://en.wikipedia.org/wiki/SQL_injection
<http://www.unixwiz.net/techtips/sql-injection.html>

[View Source](#) [View Help](#)

iv) Turn ON Interception
In Burp:
Proxy → Intercept → ON

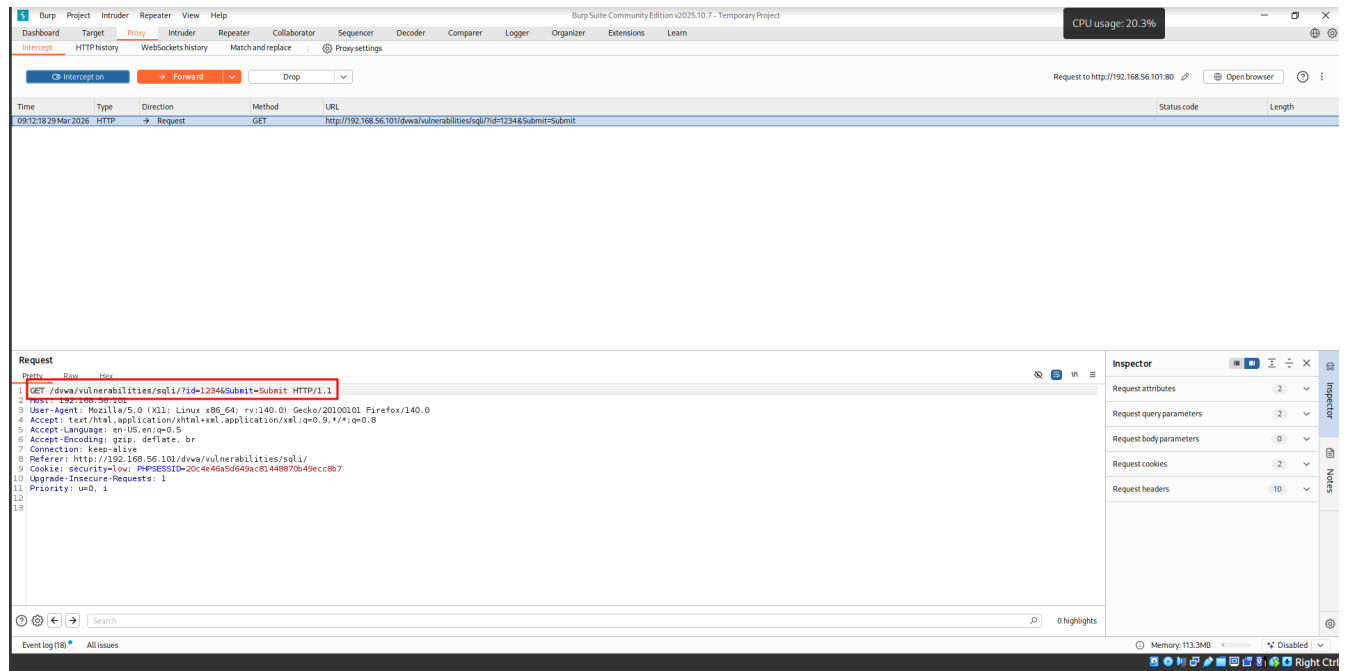


v) To see requests in Burp
Go back to the SQL injection section in the DVWA, enter any number as UserID and submit



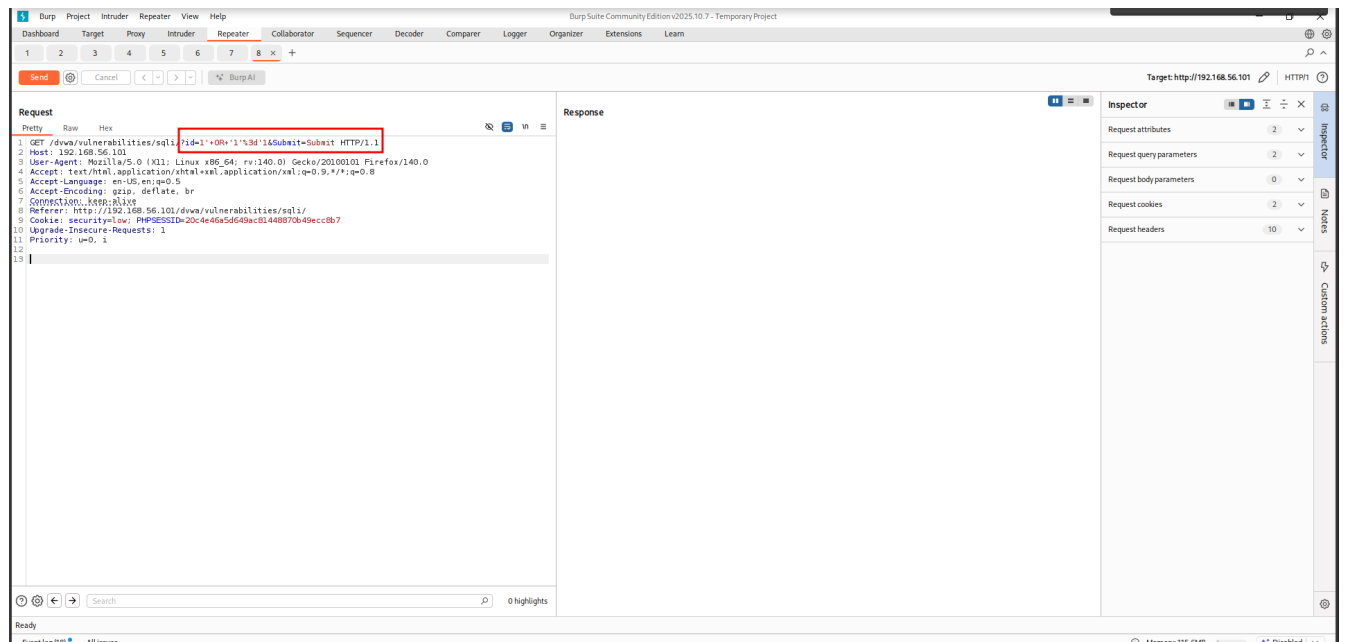
vi)Result:

Login user id from DVWA was successfully intercepted and captured in burp.

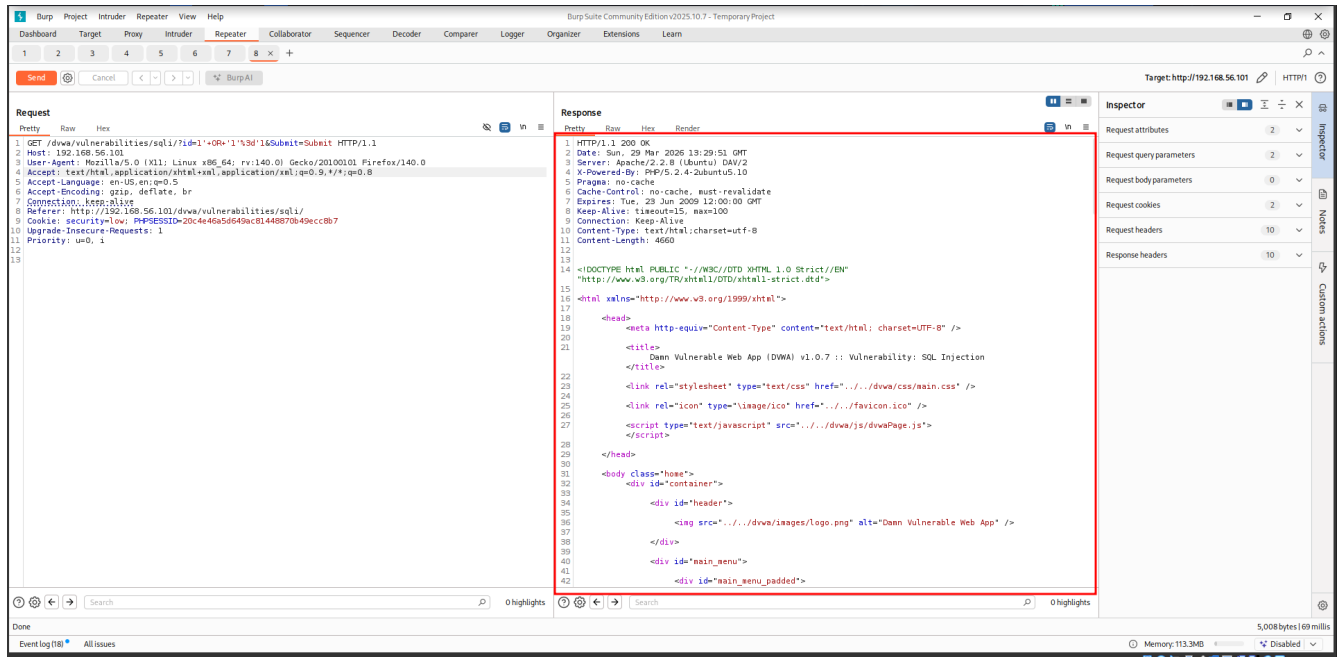


vii) Move details to Repeater and modify the payload:

Replace id=1234 with id=1' OR '1'='1 converted to id=1'+OR+'1'%3d'1
Then Send



viii) SQL Injection Successful



ix) Analyzing Request

Analysis of the Successful Request

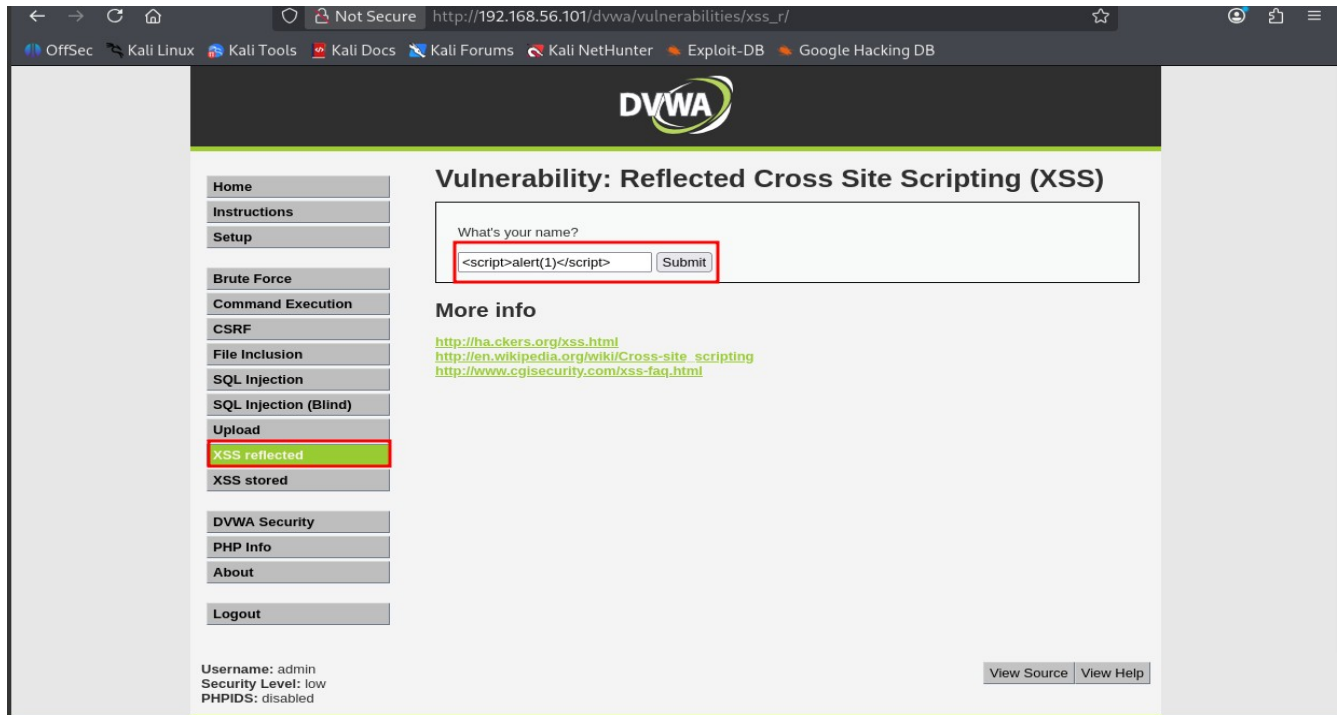
The table breaks down how my input manipulated the HTTP request to bypass the database logic

Component	Value / Content	Purpose & Impact
Method	GET	The data is sent via the URL parameters.
Resource Path	/dvwa/vulnerabilities/sqli/	The specific vulnerable endpoint you are targeting.
Vulnerable Parameter	id	The input field that is not properly sanitized by the PHP code.
Injected Payload	1' OR '1'='1	The "Tautology." By adding OR '1'='1', you make the query condition always true.
Encoded Payload	1%27+OR+%271%27%3D%271	Necessary for the server to interpret spaces and quotes as data rather than protocol delimiters.
HTTP Status Code	200 OK	Confirms the server processed the request and returned the page content.
Evidence of Success	Multiple <pre> blocks	The database dumped all rows because the WHERE clause logic was bypassed.

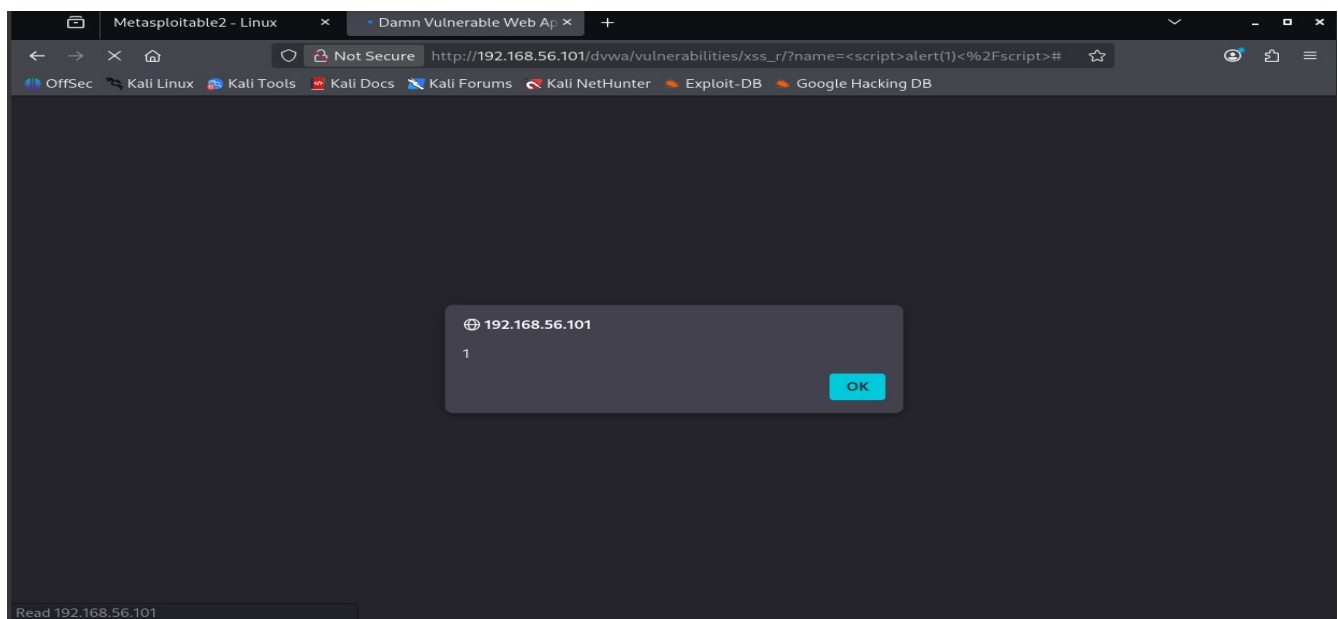
4. Weak Input Validation

Checking XSS Vulnerability

i) Try: `<script>alert(1)</script>`

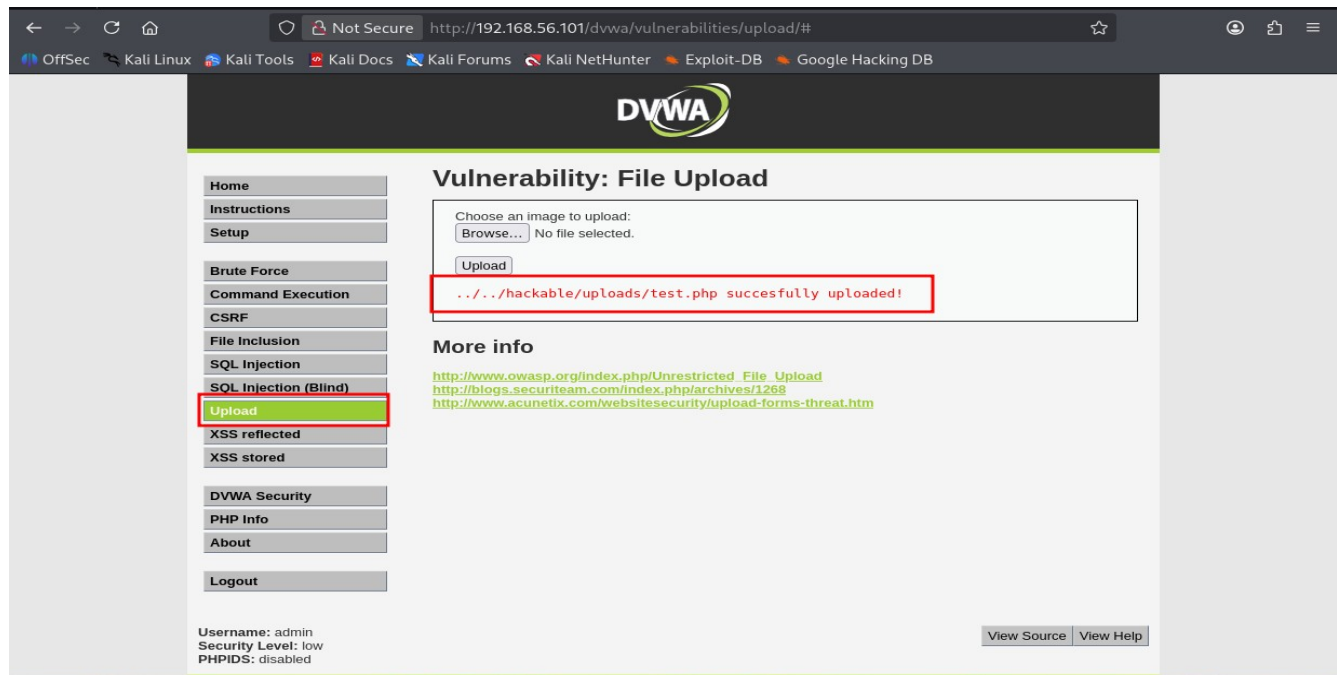


ii) XSS Vulnerability present



iii) File Upload Testing

Go to DVWA and navigate to “Upload” and then try uploading test.php file



File upload was successful

5. Vulnerability Finding Table

Vulnerability	Location	Evidence / Payload	Impact	Risk Level	Recommendation
SQL Injection	DVWA – SQL Injection module (id parameter)	Payload ' OR '1'='1 returned multiple user records including admin data.	Unauthorized data access; ability to extract sensitive database information.	High	Use prepared statements, parameterized queries, and strict input validation.
Cross-Site Scripting (XSS)	DVWA – XSS (Reflected) input field	<script>alert(1)</script> executed successfully and triggered a browser popup.	Allows execution of malicious scripts in victim's browser; high session hijacking risk.	High	Encode output, validate input, and implement a Content Security Policy (CSP).
Unrestricted File Upload	DVWA – File Upload module	test.php uploaded successfully and stored in /hackable/uploads/.	Potential Remote Code Execution (RCE) if uploaded files are executable.	High	Restrict file types (allowlist), rename uploads, and store files outside the web root.

Final Workflow:

1. Start lab
2. Open **Burp Suite**
3. Intercept requests
4. Modify inputs
5. Identify vulnerabilities